

Klasser og instanser

Download **hele** mappen `15_minigames` — ikke enkelte filer. Den hænger kun sammen, hvis det hele kommer med:

- **Opgave** — `opgaver.html`: det I løser to og to
- `api_lib` — biblioteket til *dette* forløb. Det er ikke det samme som den `api_lib`, I måske allerede har liggende i jeres eget repo. Bland dem ikke.
- **Classes demo** — `myClasses_demo`: demosiden med klassediagrammer

Opgaverne kan I kun besvare ved at læse koden. Klasserne ligger i `api_lib/myClasses.js`. Hvordan instanserne oprettes og køres i `setup` og `draw`, ligger i `myClasses_demo/index.js`. Demoen viser, hvad der sker på skærmen — svarene står i de to filer. I skal have dem åbne, mens I skriver.

I arbejder i makkerpar. Svarene gemmes i browserens `localStorage`, så I kan fortsætte ved computeren. Til det, der skal tegnes — blandt andet `flowcharts` — skal I også have en udprintet udgave. Fotografér arkene, gem som PDF og aflever.

NAVN

NAVN

Ball

Hvilke argumenter tager klassen i sin constructor, og hvad bruges de til?

```
constructor(x, y, r, img, col, xSpeed, ySpeed, bounds)
```

Hvad betyder keywordet this i en constructor?

Hvad betyder linjen

```
this.velocity = createVector(xSpeed || 0, ySpeed || 0)
```

Hvilke metoder har klassen?

I metoden constrain finder I denne linje — hvad gør den?

```
if(b[0] && this.position.x < r){  
  this.position.x = r  
  this.velocity.x *= -1  
}
```

Forklar metoden hit

```
hit(anotherBall){  
  var b = anotherBall  
  var totalR = (this.diam + b.diam) / 2  
  var d = dist(this.position.x, this.position.y, b.position.x, b.position.y)  
  return d <= totalR  
}
```


Hvad betyder linjerne

```
this.mass = this.diam  
this.fixed = false
```


Hvad er forskellen på hit(anotherBall) og bounce(other)?

JumpingBall

Hvad betyder linjen

```
class JumpingBall extends Ball
```


Hvad betyder linjen

```
super(x, y, r, img, col, 0, 0, bounds || [false, false, false, true])
```


Forklar metoden update

```
update(){  
  this.velocity.add(gravity)  
  this.velocity.y *= friction  
  super.update()  
}
```

Forklar metoden jump

```
jump(){  
  this.velocity.y -= this.jumpForce  
}
```

SpringBall

Klassen har en lidt speciel constructor. Hvad betyder linjen

```
super(x, y, r, img, col, 0, 0, bounds)
```

Hvad er forholdet mellem this.start og this.maxDragDist?

```
this.start = createVector(x, y)  
this.maxDragDist = maxDragDist
```

Forklar metoden drag() linje for linje

```
drag(){
  var mouse = createVector(mouseX, mouseY)
  mouse.sub(this.start)
  if(mouse.mag() > this.maxDragDist) mouse.setMag(this.maxDragDist)
  this.position = this.start.copy()
  this.position.add(mouse)
  this.isDragging = true
}
```


Forklar metoden launch() linje for linje

```
launch(){
  this.velocity = this.start.copy()
  this.velocity.sub(this.position)
  this.velocity.mult(0.15) // 0.15 dæmper farten, så bolden ikke flyver for hurtigt
  this.isFlying = true
  this.isDragging = false
}
```


Forklar metoden update

```
update(){
  if(!this.isFlying) return
  this.velocity.add(gravity)
  super.update()
}
```


countHit

Når to bolde rammer hinanden, har vi brug for en måde at tælle ET point. Klassen Ball har metoden `hit(anotherBall)`, som alle andre bolde nedarver. Den returnerer `true`, hvis den anden bold er inden for dens radius. Koden ser sådan her ud:

MYCLASSES.JS

```
hit(anotherBall){
  var b = anotherBall
  var totalR = (this.diam + b.diam) / 2
  var d = dist(this.position.x, this.position.y, b.position.x, b.position.y)
  return d <= totalR
}
```

Men `hit()` er sand i hvert eneste frame, så længe de to stadig overlapper. Hvis vi tæller dér, får vi mange point for ét sammenstød. Derfor har skitsen `countHit`, som tager begge instanser og holder øje med, om de allerede rørte hinanden i forrige frame:

INDEX.JS

```
countHit(spring, other)

function countHit(a, b){
  var h = a.hit(b)
  if(h && !a.wasHit){
    hits++
    select('#g-points').html(hits)
  }
  a.wasHit = h
}
```

Forklar linjerne i `countHit`.

Livscyklus

Hvis vi følger en `SpringBall` fra den oprettes, til den skydes af og til sidst forsvinder, kan vi se på de her steder i koden:

INDEX.JS

```

// I setup()
spring = new SpringBall(400, 420, 80, null, 120, '#87ceeb')

// I draw()
if(mouseIsPressed && (
  spring.isDragging ||
  dist(mouseX, mouseY, spring.position.x, spring.position.y) < spring.diam / 2
)){
  spring.drag()
}
spring.update()
spring.constrain()
countHit(spring, other)
spring.bounce(other)
spring.show()
if(spring.isFlying && (
  spring.position.x < 0 ||
  spring.position.x > width ||
  spring.position.y > height
)){
  spring.reset()
}

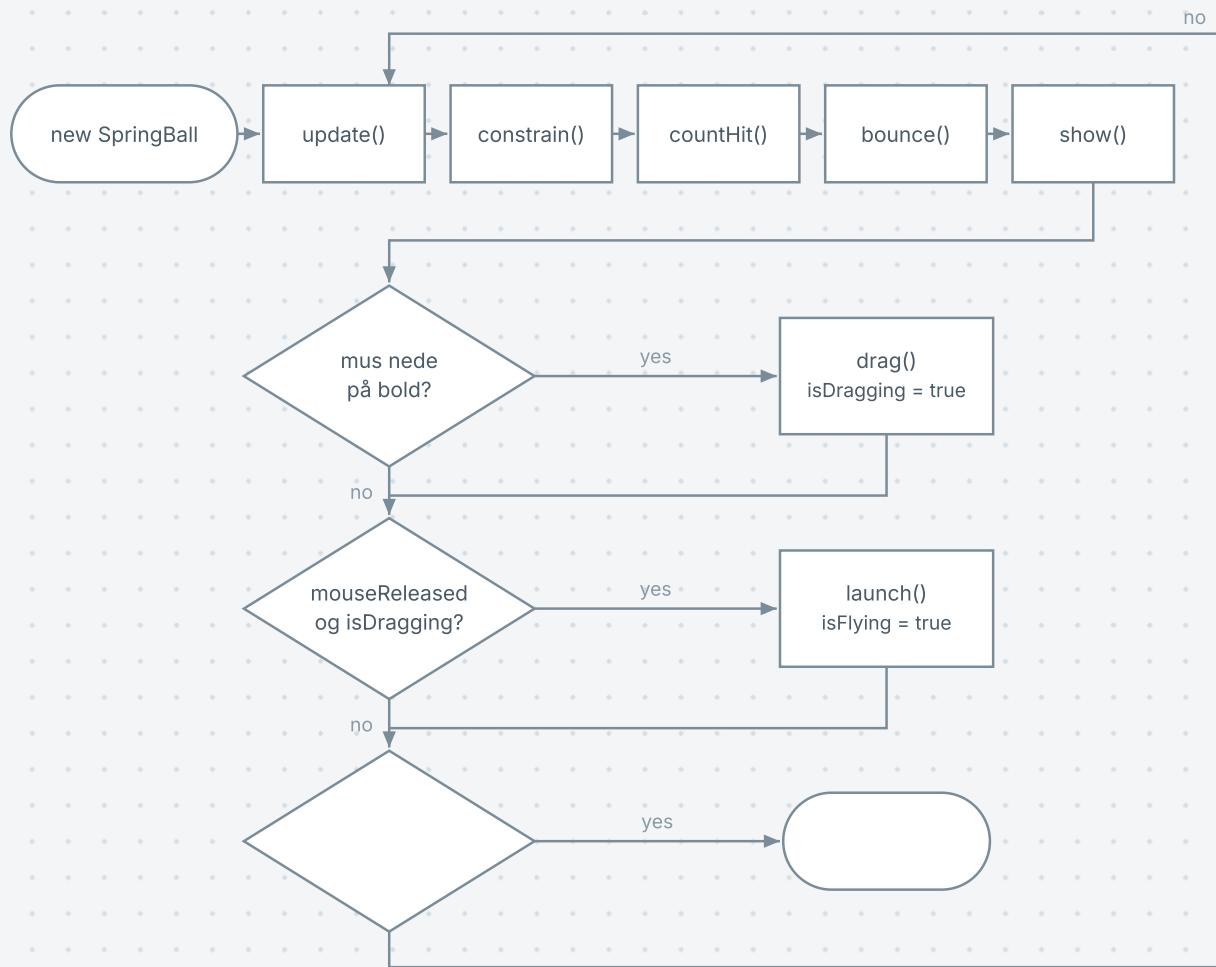
// I mouseReleased()
if(currentPage == '#page3' && spring.isDragging) spring.launch()

```

Et godt flowchart viser de processer og valg der indgår i en hvilken som helst algoritme. Et flow starter og slutter altid med runde hjørner — processer er firkantede, og valg er diamantformede. Der går ALTID to muligheder ud fra et valg: ja eller nej i daglig tale.

Se om I kan afslutte flowchartet nedenfor — hvad er det sidste valg der indgår i en SpringBall-livscyklus, og hvad medfører det?

SpringBall livscyklus



MovingBall

Hvad er `this.heading`, og hvad gør `left()`, `right()`, `up()` og `down()` ved den?

Hvad betyder linjen

```
this.velocity.set(this.heading.x * this.speed, this.heading.y * this.speed)
```


Forklar metoden update linje for linje

```
update() {  
  this.heading.normalize()  
  super.update()  
}
```

Hvad tegner show() ud over selve bolden?

```
var tip = this.heading.copy().setMag(this.diam)  
line(this.position.x, this.position.y, this.position.x + tip.x, this.position.y + tip.y)
```

Hvis nogle instanser af MovingBall skal kunne køre fremad hele tiden: hvordan vil I implementere det? Kig på clearDrive og forward.

Livscyklus

Hvis vi følger en MovingBall fra den oprettes, kan vi se på de her steder i koden:

INDEX.JS

```

// I setup()
move = new MovingBall(160, 320, 80, null, 6, '#3cb371', [true, true, true, true])

// I draw()
clearDrive(move)
steer(move)
move.update()
move.constrain()
countHit(move, other)
move.bounce(other)
move.show()

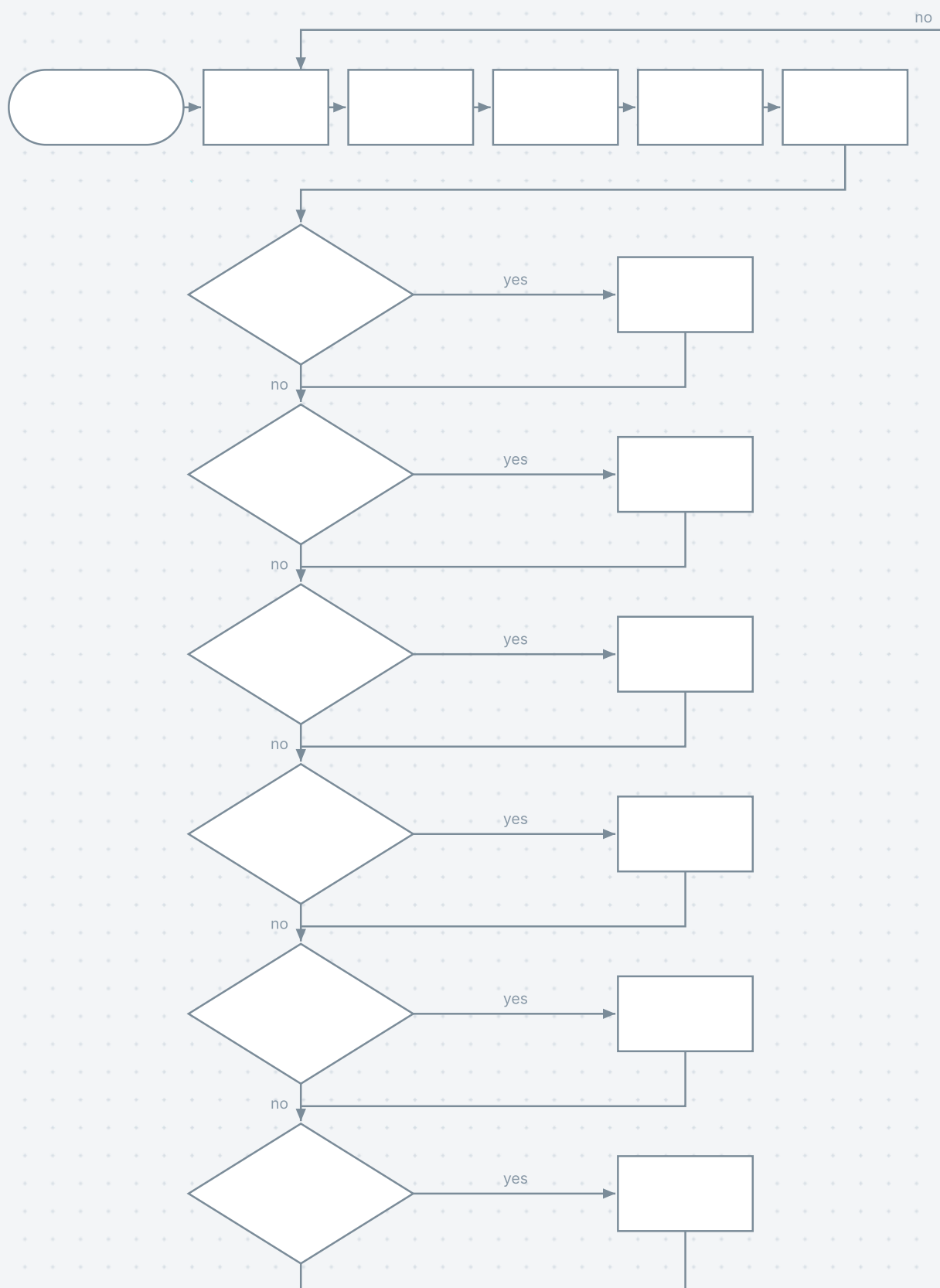
// I draw() - steer()
function steer(b){
  if(keyIsDown(65)) b.left()
  if(keyIsDown(68)) b.right()
  if(keyIsDown(87)) b.up()
  if(keyIsDown(83)) b.down()
  if(keyIsDown(UP_ARROW)) b.forward()
  if(keyIsDown(DOWN_ARROW)) b.back()
}

function clearDrive(b){
  if(keyIsDown(UP_ARROW) || keyIsDown(DOWN_ARROW)){
    b.didBounce = false
    return
  }
  if(b.didBounce) return
  b.velocity.set(0, 0)
}

```

Udfyld kasserne i flowchartet ud fra koden.

MovingBall livscyklus



ShootingBall

Hvorfor extends MovingBall og ikke extends Ball?

Hvad er this.bullets?

```
this.bullets = []
```


Forklar metoden shoot() — især de to første argumenter til new Bullet

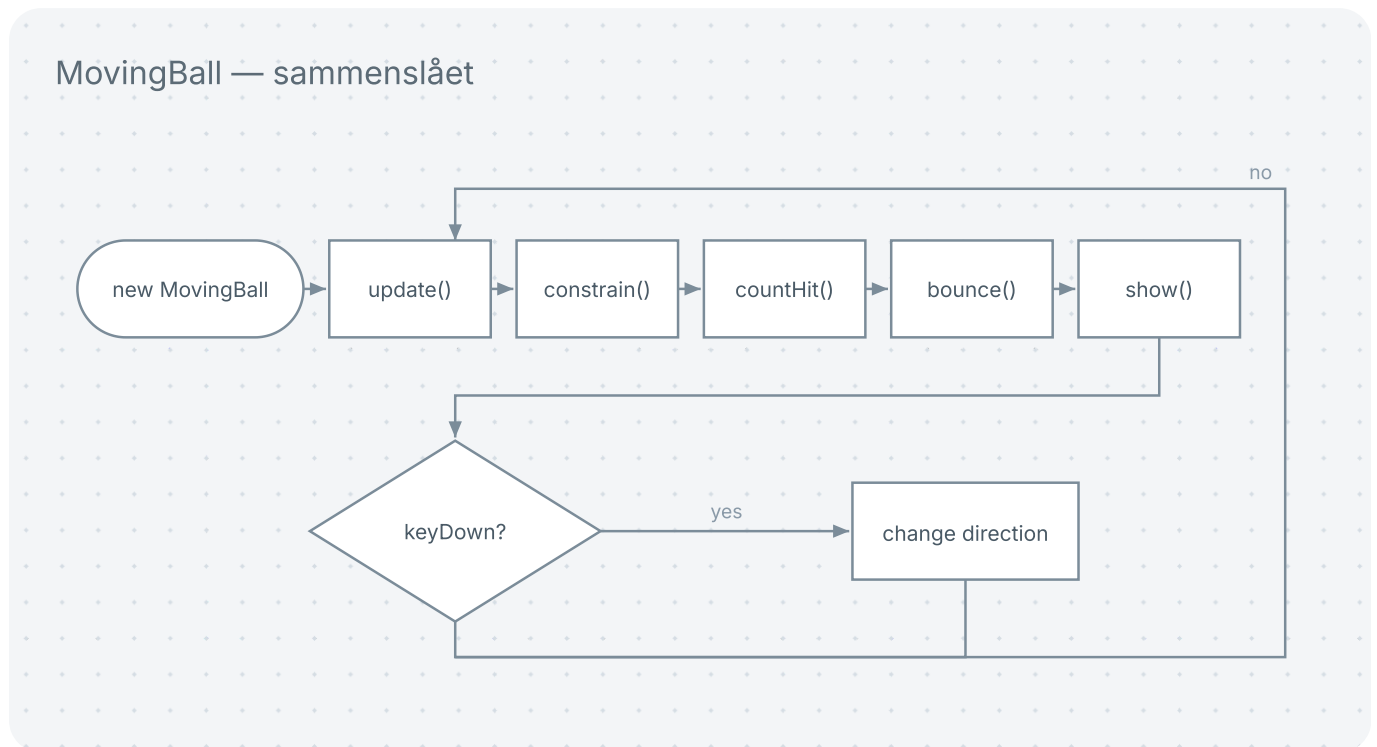
```
this.bullets.push(new Bullet(  
  this.position.x + this.heading.x * this.diam,  
  this.position.y + this.heading.y * this.diam,  
  this.heading,  
  this.bulletSpeed,  
  this.col  
))
```


Forklar metoden update

```
update(){  
  super.update()  
  this.bullets.map(b => b.update())  
  this.bullets = this.bullets.filter(b => !b.isGone())  
}
```


Livscyklus

De seks taster i `steer()` kan slås sammen til ét valg og én proces:

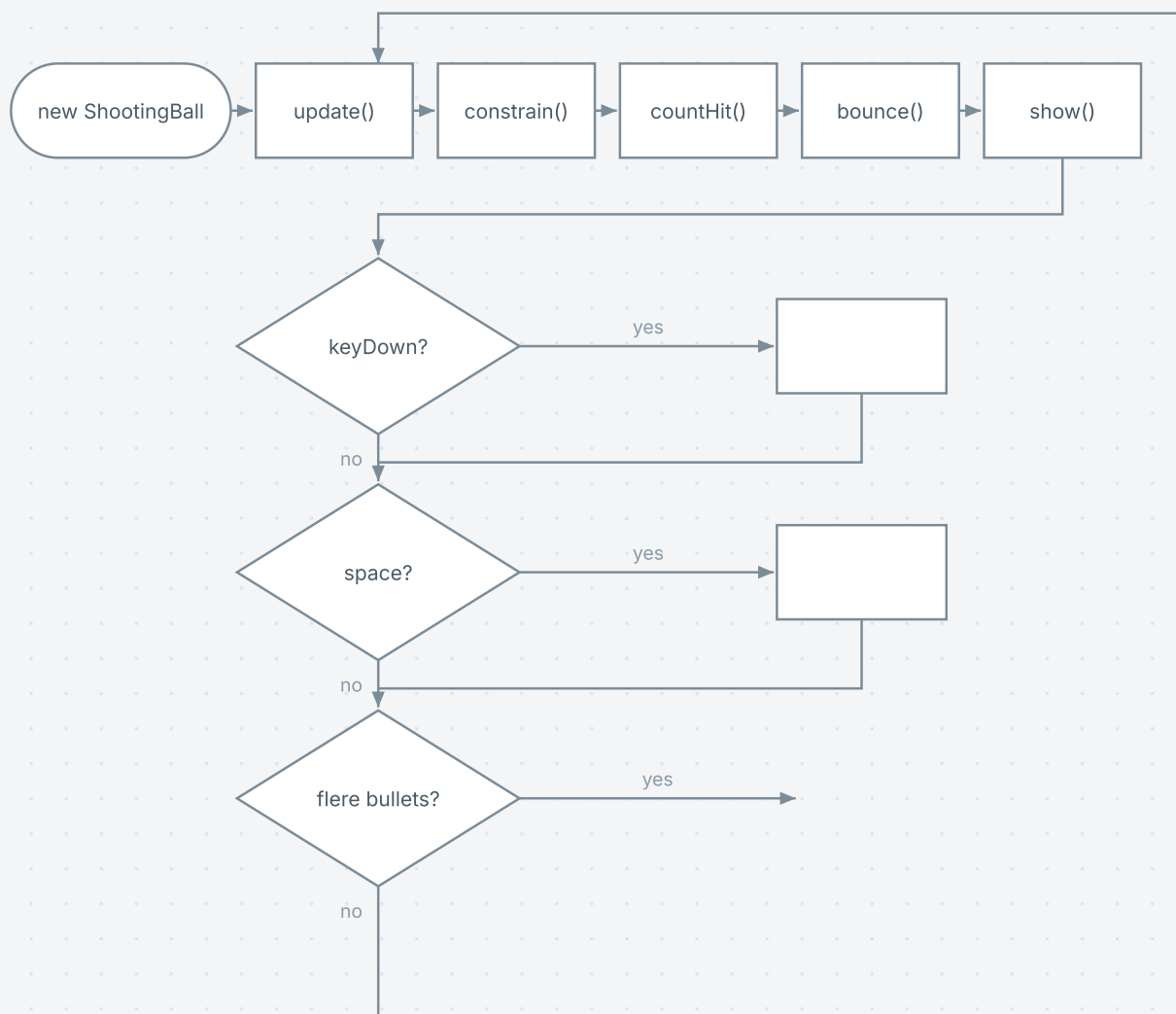


I skal lave et flowchart til en `ShootingBall`. Den er lige til `MovingBall`, men har det ekstra lag at den hele tiden vedligeholder sit array med bullets. De bullets skal passes som en del af dens livscyklus.

I hvert frame gør `ShootingBall` det samme som `MovingBall`. Derudover løber den bullets igennem: opdaterer hver enkelt, lader dem bounce, og fjerner dem der er væk. Hvis der skydes, kommer der en ny bullet på arrayet.

Udfyld kasserne ved `keyDown?` og `space?`. Fra flere bullets? tegner I selv de kasser og valg I synes der skal til.

ShootingBall livscyklus



Bullet

Hvorfor extends Ball og ikke MovingBall?

Hvad betyder de sidste to argumenter til super?

```
super(x, y, 16, null, col || '#111', heading.x * speed, heading.y * speed)
```

Forklar metoden isGone

```
isGone(){  
  return this.position.x < 0 || this.position.x > width || this.position.y < 0 || this.position.y >  
  height  
}
```

Livscyklus

Hvis vi følger en Bullet fra den oprettes, til den forsvinder, kan vi se på de her steder i koden:

INDEX.JS OG MYCLASSES.JS

```
// I ShootingBall.shoot()  
this.bullets.push(new Bullet(  
  this.position.x + this.heading.x * this.diam,  
  this.position.y + this.heading.y * this.diam,  
  this.heading,  
  this.bulletSpeed,  
  this.col  
))  
  
// I ShootingBall.update()  
this.bullets.map(b => b.update())  
this.bullets = this.bullets.filter(b => !b.isGone())  
  
// I draw()  
shoot.bullets.map(b => {  
  countHit(b, other)  
  b.bounce(other)  
})  
  
// I ShootingBall.show()  
this.bullets.map(b => b.show())
```

Tegn flowchartet.

Bullet livscyklus

Firestore

Hvilke argumenter tager klassen i sin constructor, og hvad bruges `this.ref` til?

```
constructor(collection) {  
  this.ref = db.collection(collection)  
}
```

Hvad gemmer metoden `save`?

```
save(name, points) {  
  this.ref.add({  
    name: name,  
    points: points,  
    timestamp: firebase.firestore.FieldValue.serverTimestamp()  
  })  
}
```

Hvordan kunne I skrive `save` om, så den kan gemme flere eller andre værdier end bare `name` og `points`?

Hvad gør metoden `listen`?

Hvad er forskellen på `remove(id)` og `clear()`?
